
Supervised Learning

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

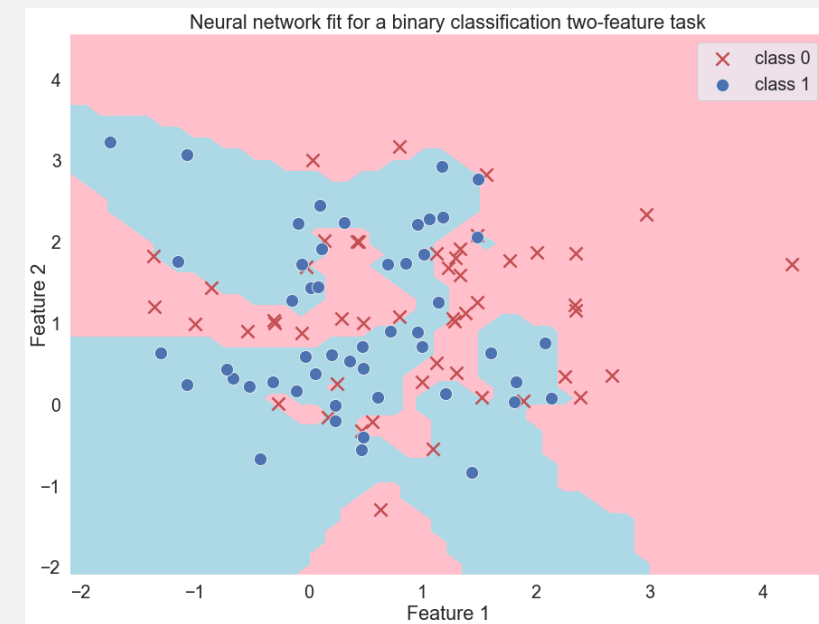
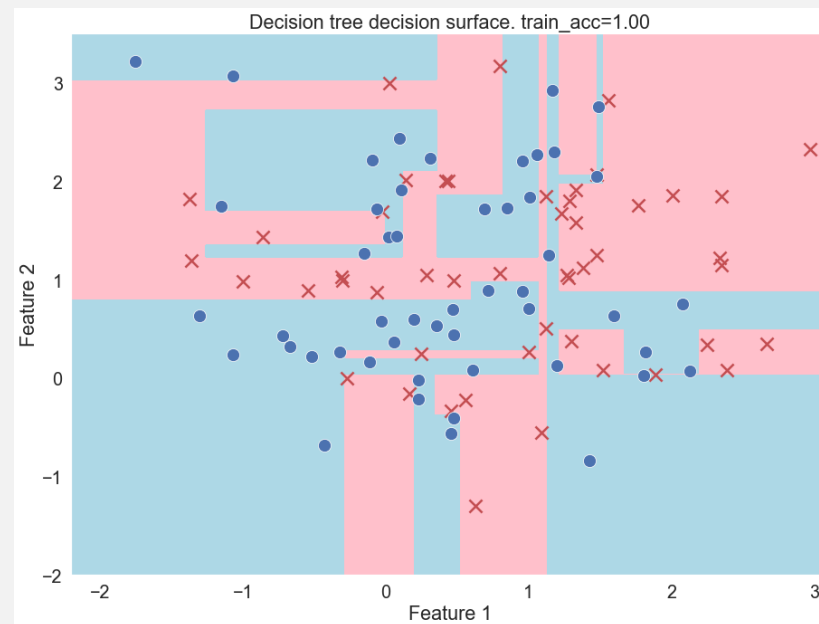
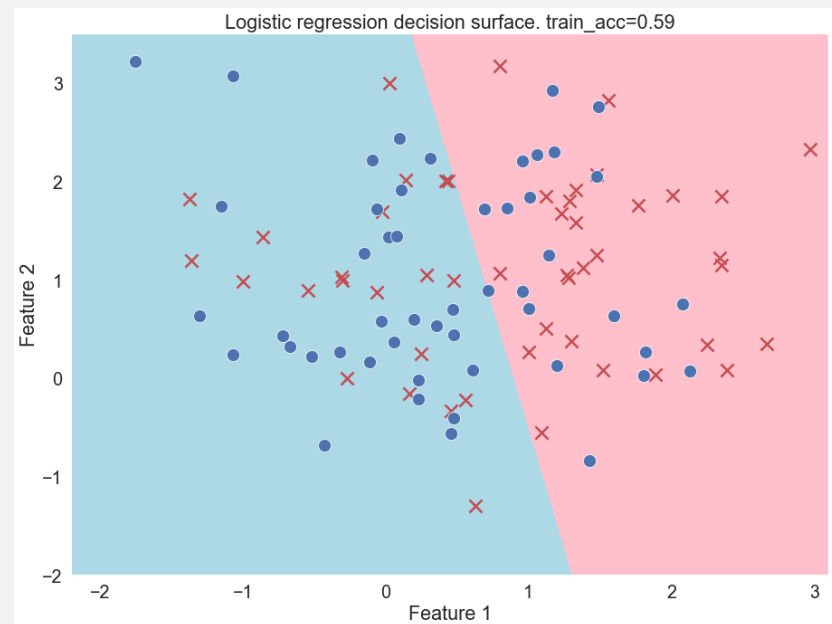
Outline

- Formalized Supervised Learning Setup
 - Supervised learning sampling process
 - Loss functions and risk
 - Empirical risk minimization
 - The optimal classifier: Baye's classifier
 - Generative vs discriminative approach
- Practical considerations:
 - Splitting Dataset into Training and Testing
 - Data normalization techniques
 - Handling missing data

Supervised Learning Goal

- Given a **labelled** dataset set $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$, the goal in supervised learning is to use the data to train a machine learning model that is able to **generalize** to **unseen** instances.
- **Generalization:** The ability of a model to **correctly** predict the label of *new* instances that it did not train on.

- Every ML algorithm will make assumptions on the *form* the relationship between input x and label y .
- Here are some famous ML algorithms on their assumptions:
 - Linear/Logistic regression: assumes y is a linear function of the features.
 - Decision tree: assume y is axis-aligned rectangles decision boundaries.
 - Neural network: assume y is highly non-linear function of the features.
- See the following 2-dimensional classification training dataset where Logistic regression, decision tree, and neural network is fit. Notice that *form* of these functions. Notice the accuracy of the training set.
- Using very strong/complex models can easily allow you to get an excellent fit on the training set. However, this can lead to **overfitting**.



Formalized Supervised Learning

Recall Helpful Matrix Notation

- We can represent a dataset $\{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ with m examples and n features as a feature matrix $X \in \mathbb{R}^{m \times n}$ and a label vector $Y \in \mathbb{R}^m$

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} \quad \bigg| \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

- X is referred to as the feature or data matrix. Y is the label or output vector.

Supervised Learning Setup [1]

The supervised learning problem consists of:

- **Labeled Dataset:** A dataset set $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ consists of m samples.
- **Input features:** Each x_i is a vector of n dimensions.
- We'll use notation x_{ij} to denote j 'th feature of i 'th sample.
- Each features has a domain of possible values. For example:

$x_{ij} \in \mathbb{Z}$ j th element takes on integer numbers $\{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$

$x_{ij} \in \mathbb{Z}^+$ j th element takes on positive integer numbers $\{1, 2, 3, \dots\}$

$x_{ij} \in \mathbb{R}$ j th element takes on real-number values from $(-\infty, +\infty)$

$x_{ij} \in \mathbb{R}^+$ j th element takes on non-negative real-number values from $0, +\infty$

$x_{ij} \in \{0, 1\}$ j th element takes on values from the set of size 2 $\{0, 1\}$

Supervised Learning Setup [2]

- **Output label:** The output label y_i can be a single value (scalar) or can be a vector. For simplicity, we will assume it is a single value for now.
- Similar to the input, the output label y_i has a range of possible values. For example:

$y_i \in \mathbb{R}$ output label takes on real-number values from $(-\infty, +\infty)$

$y_i \in \mathbb{Z}^+$ output label takes on positive integer numbers $\{1, 2, 3, \dots\}$

$y_i \in \{0, 1\}$ output label takes on values from the set of size 2 $\{0, 1\}$

Supervised Learning Sampling Process [1]

- It is typical to assume that every feature in our instance vector x_i is distributed according to *some* probability distribution. For example:

$x_{ij} \sim \mathcal{N}(\mu, \sigma^2)$ follows a normal distribution with mean μ and standard deviation σ

$x_{ij} \sim \text{Uniform}(0, 1)$ follows a uniform distribution from 0 to 1

$x_{ij} \sim \text{Bernoulli}(p)$ used for features with just 2 values. $\text{Prob}[x_{ij} = 1] = p$

$x_{ij} \sim \mathcal{D}_j$ feature j follows distribution $\mathcal{D}_j$

- Note: we are saying the that $x_{1j}, x_{2j}, \dots, x_{nj}$ follow the same distribution $\mathcal{D}_j$; another way of saying this is that these are samples of a random variable $X_j \sim \mathcal{D}_j$
- We can also say that the vector $x_i \sim D$ where D is a vector distribution. You can explicitly write out D to state the distribution for the n features. That is, $x_{i1} \sim D_1, x_{i2} \sim D_2, \dots, x_{in} \sim D_n$ where D_i is some valid scalar random variable distribution.

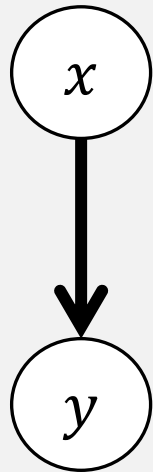
Supervised Learning Sampling Process [2]

- In supervised learning, we assume that a sample (x, y) is drawn/sampled from some underlying distribution.
- So, given a dataset $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, each (x_i, y_i) is sampled from some underlying distribution.
- We say that the samples are **independently and identically distributed (iid)**
 - Independent: the samples are independent of each other. Each sampling does not influence another.
 - Identically distributed: each instance is sampled from the same underlying distribution D
- One simple formulation is to assume that the input $x \sim D$ is sampled from a distribution D , and then a labelling function $y_i = f(x_i)$ computes the label.
- A more flexible probabilistic formulation is to *assume* how to model the joint $p(x, y)$
 - We have three options which depends on the order of sampling.
 - Option 1: $p(x, y) = p(x)p(y|x)$ The order is to sample $x \sim D_1$, then sample $y | x \sim Q_1$ where Q_1 is distribution of y *given* features x .
 - Option 2: $p(x, y) = p(y)p(x|y)$ The order is to sample $y \sim D_2$, then sample $x | y \sim Q_2$
 - Option 3: model the full joint $p(x, y)$ that is $(x, y) \sim D_3$

Supervised Learning Sampling Process – Option 1: $p(x, y) = p(x)p(y|x)$

- Thus, the labelled dataset $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ are sampled from some underlying distribution.
- The sampling process involves sampling the features x , then sampling the label. We can visualize this as a Bayesian network.

Probabilistic dependency graph
(or Bayesian Network)



$$p(x, y) = p(x)p(y|x) \text{ where } x \sim D \text{ and } y|x \sim Q$$

First, x_i is sampled from some vector distribution $x \sim D$.

Then, y_i given x_i is sampled from some conditional distribution $y|x \sim Q$

This is done for each $i = 1, 2, \dots, m$

Supervised Learning Sampling Process – Option 1: $p(x, y) = p(x)p(y|x)$

- Consider the following example. You **know** the following:

$x_i \in \mathbb{R}^2$ where $x_{i1} \sim \text{Normal}(\mu = 10, \sigma = 5)$ and $x_{i2} \sim \text{Uniform}(0, 1)$.

$$y_i = (2x_{i1} - x_{i2})$$

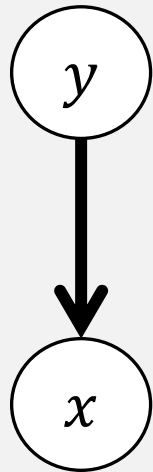
- How can you sample $m = 1000$ dataset from this setup?

```
1 import numpy as np
2
3 dataset = []
4 for i in range(1000):
5     x1 = np.random.normal(loc=10.0, scale=5.0)
6     x2 = np.random.uniform(low=0.0, high=1.0)
7
8     y = (2*x1 - x2)
9
10    features = [x1, x2]
11    sample = (features, y)
12    dataset.append(sample)
```

Supervised Learning Sampling Process – Option 2: $p(x, y) = p(y)p(x|y)$

- Thus, the labelled dataset $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ are sampled from some underlying distribution.
- The sampling process involves sampling label y , then sampling $x|y$. We can visualize this as a Bayesian network.

Probabilistic dependency graph
(or Bayesian Network)



$$p(x, y) = p(y)p(x|y)$$

First, y_i is sampled.

Then, x_i given y_i is sampled.

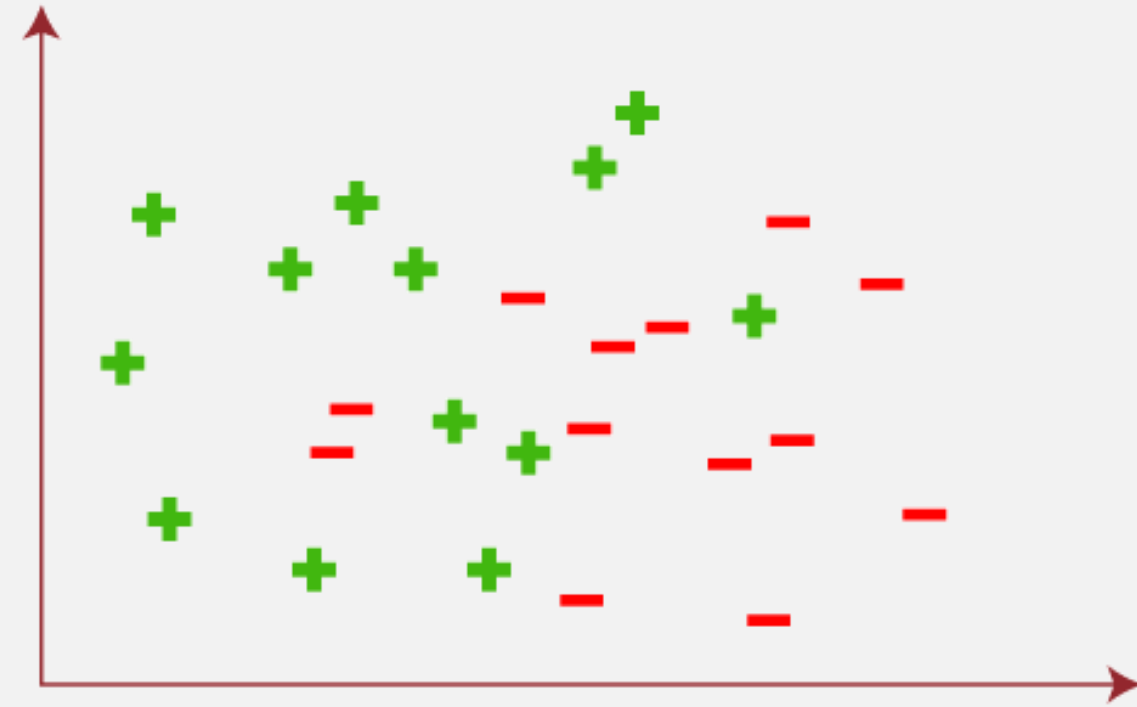
This is done for each $i = 1, 2, \dots, m$

ML algorithm, model, hypothesis, and hypothesis space

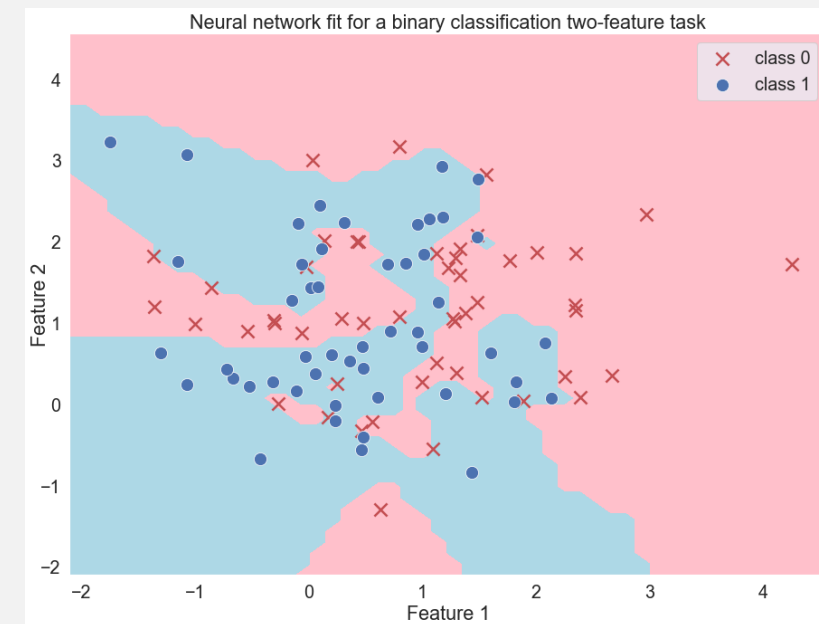
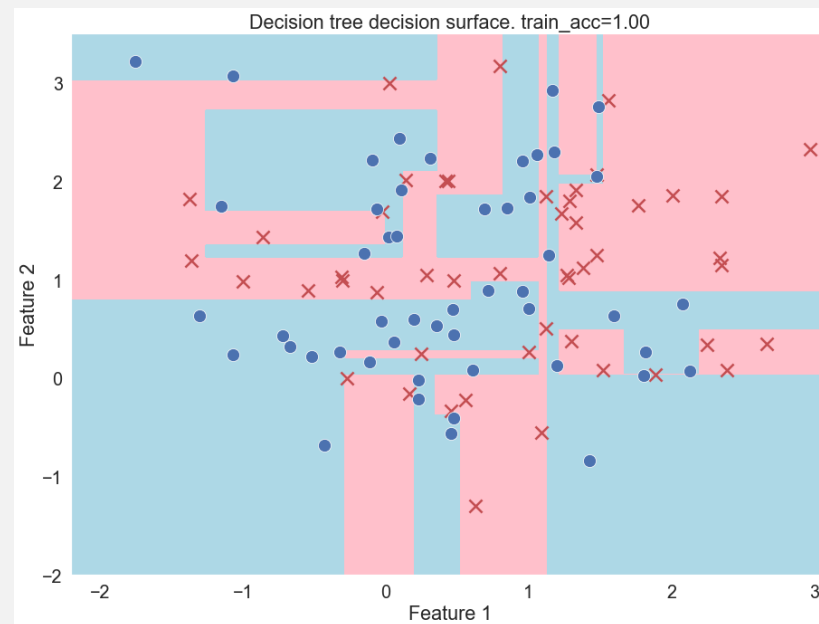
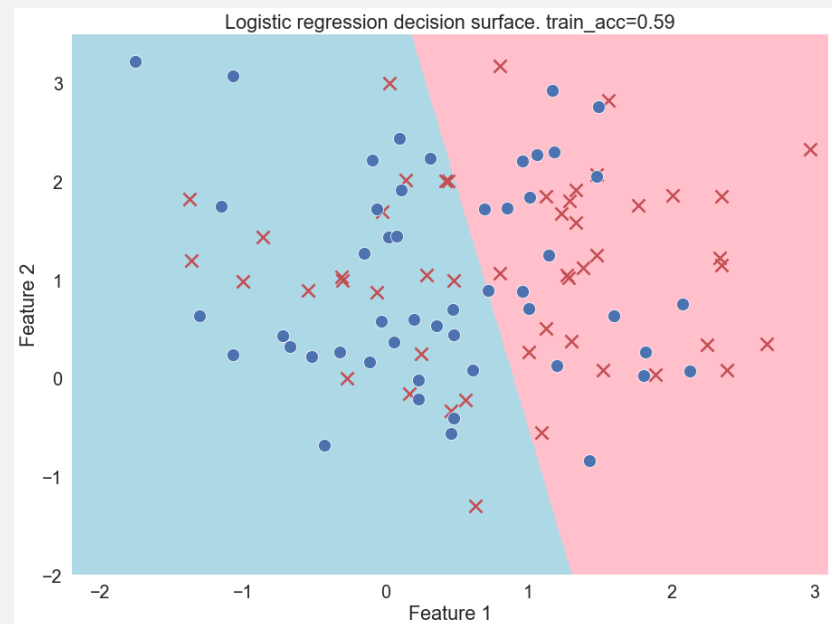
- It is noteworthy to differentiate between machine learning *algorithm* and machine learning *model*.
- An ML algorithm is trained on a dataset to produce a model.
- A model is a function h that takes in a feature vector to predict the label $\hat{y} = h(x)$. Other names for model is *hypothesis function*, *classifier*, *predictor*.
- For example, linear regression is an ML algorithm. After training on a dataset, it produces a linear model.
- Every ML algorithm makes assumptions on the *form* of the hypotheses it considers.
 - This is called the *hypothesis space or hypothesis set* of the ML algorithm. Typically denoted as $\mathcal{H}$
 - The assumptions restrict the hypotheses under consideration.
 - This restriction is important for generalization (finding a model that performs well in production).
 - In general, an ML algorithm that has a flexible hypothesis space is prone to *overfitting* (see later slides for formal definition).

Hypothesis Space Example

- Consider the binary label 2-feature dataset in the figure.
- Consider an ML algorithm called RectLearner.
 - This algorithm learns the rectangle that best fits a binary labelled dataset.
 - Instances within the rectangle are labelled negative class, and instance outside are labelled the positive class.
 - What is the hypothesis space for this algorithm?
- To formally define the hypothesis space of the RectLearner, we notice that a rectangle is defined by two points (x_{tl}, y_{tl}) and (x_{br}, y_{br}) (top-left and bottom-right).
- Thus, the hypothesis space is the set of all rectangles defined by two points:
$$\mathcal{H} = \{(x_{tl}, y_{tl}), (x_{br}, y_{br}) \mid (x_{tl}, y_{tl}, x_{br}, y_{br}) \in \mathbb{R}^4\}$$
- Homework: what about an ML algorithm that learns circles?



- Every ML algorithm will make assumptions on the *form* the relationship between input x and label y .
- Here are some famous ML algorithms on their assumptions:
 - Linear/Logistic regression: assumes y is a linear function of the features.
 - Decision tree: assume y is axis-aligned rectangles decision boundaries.
 - Neural network: assume y is highly non-linear function of the features.
- See the following 2-dimensional classification training dataset where Logistic regression, decision tree, and neural network is fit. Notice that *form* of these functions. Notice the accuracy of the training set.



Loss function

- So, every ML algorithm will have a hypothesis space $\mathcal{H}$ that it considers.
- Given a hypothesis space $\mathcal{H}$ how to find the best $h \in \mathcal{H}$?
- Given a sample (x_i, y_i) and hypothesis h , we can use a **loss function** to compute the loss between a prediction $h(x_i)$ and the true label y_i .
- Let's denote this loss function as $\ell(h(x_i), y_i)$.
- Three common loss functions (there are many more):

$\ell(h(x_i), y_i) = |h(x_i) - y_i|$ absolute value difference; known as L-1 norm.

$\ell(h(x_i), y_i) = (h(x_i) - y_i)^2$ squared value difference; known as L-2 norm.

$\ell(h(x_i), y_i) = \begin{cases} 0 & \text{if } h(x_i) == y_i \\ 1 & \text{if } h(x_i) \neq y_i \end{cases}$ known as 0-1 loss.

Training an ML Algorithm – Empirical Risk Minimization (ERM)

- We can use a loss function ℓ with the dataset S to find $h \in \mathcal{H}$ that gives the smallest error.
- Given a labelled dataset S , hypothesis h , and loss function ℓ . We can compute the **empirical loss** as follows:

$$\begin{aligned} EmpLoss(S, h) &= \frac{1}{|S|} \sum_{(x_i, y_i) \in S} \ell(h(x_i), y_i) \\ &= \frac{1}{m} \sum_{i=1}^m \ell(h(x_i), y_i) \end{aligned}$$

This is the avg loss of h on the labelled dataset S .

- So, given an ML algorithm with hypothesis space $\mathcal{H}$, how do we find the best hypothesis?
- We select the hypothesis that gives the minimum empirical loss. This is *training*, formally known as **empirical risk minimization (ERM)**. Risk and loss are used interchangeably.

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(S, h)$$

h^* is called **empirical loss/risk minimizer**

Empirical Risk vs True Risk (generalization)

We are using $p(x, y) = p(x)p(y|x)$
Note that $p(x, y) = p(y)p(x|y)$ also works

- Recall that the goal of supervised learning is generalization.
- The real way to achieve generalization is to minimize the true risk/loss:

$$TrueLoss(h) = E_{x \sim \mathcal{D}, y|x \sim \mathcal{Q}}[\ell(h(x), y)]$$

This is the average loss of h over all possible (x, y) samples.

$$= \int_{x, y} \ell(h(x), y) p(y|x) p(x) dx dy$$

This is explicitly an integral over the possible samples (x, y) , weighed by their probabilities. Some instances may appear more frequently than others.

$$h^* = \arg \min_{h \in \mathcal{H}} TrueLoss(h)$$

- So, if this achieves generalization, why don't we just minimize the true loss?
- Because in practice, we do not know the underlying distributions, so, we cannot compute this expectation.
- However, we can compute an *estimate* of the true loss. This is the empirical loss.

Empirical Risk vs True Risk (generalization)

$$TrueLoss(h) = E_{x \sim \mathcal{D}, y|x \sim \mathcal{Q}}[\ell(h(x), y)] \qquad EmpLoss(S, h) = \frac{1}{|S|} \sum_{(x_i, y_i) \in S} \ell(h(x_i), y_i)$$

- Where did the samples of S come from? From the underlying distribution.
- How good of an estimate is the empirical loss? Recall the law of large numbers from statistics.
 - The sample average of n samples $s_1, s_2, \dots, s_n$ converges to the expectation (mean) $E[s]$ as $n \rightarrow \infty$

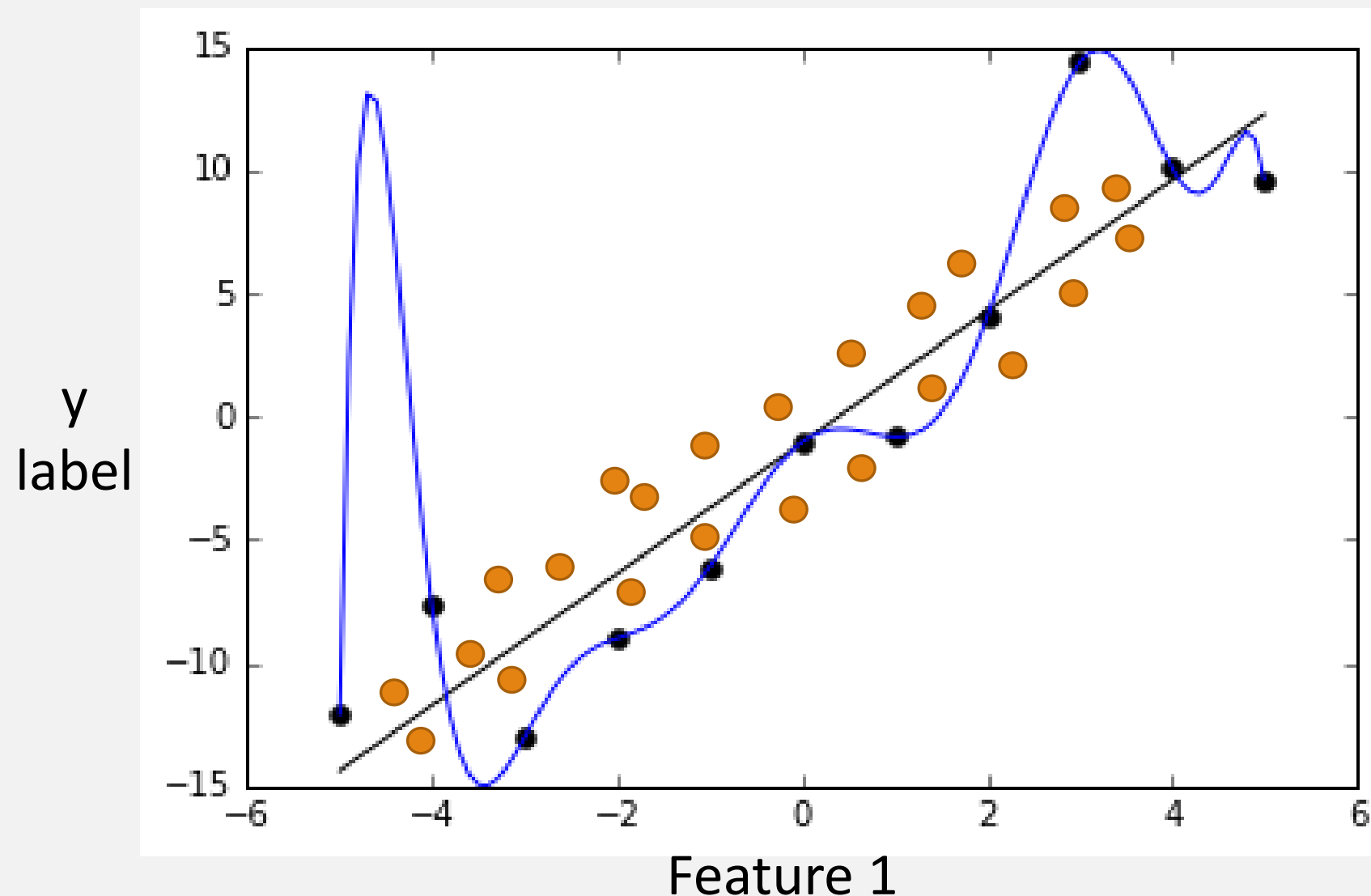
For any h , $EmpLoss(S, h) \rightarrow TrueLoss(h)$ as $|S| \rightarrow \infty$

$$\arg \min_{h \in \mathcal{H}} EmpLoss(S, h) \rightarrow \arg \min_{h \in \mathcal{H}} TrueLoss(h) \text{ as } |S| \rightarrow \infty$$

- So, what we can say is that empirical loss minimization is a sound strategy.
 - As the size of the dataset increases, the empirical loss approaches the true loss.
 - With large dataset, minimizing empirical loss is approximate to minimizing true loss.
- What can happen if dataset is not large enough?

Overfitting [1]

An ML algorithm with $\mathcal{H} = \{h_{blue}, h_{black}\}$



1. Suppose we have a learning problem with 1 input feature. We have a labelled dataset S. We plot the points (x, y) that are in S. Assume we use distance to compute the loss.

2. Suppose S are the black points. This is the data we have.

3. The orange points represents the underlying data. We don't have this data. It is hidden/unseen.

4. The blue hypothesis clearly predicts the points in S with zero empirical loss. But it has high true loss. This is a case of **overfitting**.

5. However, the black line hypothesis is a **better fit** for the true loss.

Overfitting [2]

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(S, h)$$

- In the previous plot. We saw that the blue hypothesis h^* clearly minimizes the loss on S (empirical loss). But has high loss on the underlying unseen data distribution (true loss). This happens when the machine learning algorithm memorizes or **overfits** the data it trained on.
- Overfitting generally occurs because:
 - During training, we find the hypothesis h^* with minimum EmpLoss on the specific training dataset S . This is not necessarily the minimizer of the TrueLoss. We say that empirical loss minimizer h^* is **biased** towards the training dataset.
 - There are two sources of error for $EmpLoss(S, h^*)$: the hypothesis space $\mathcal{H}$ and the size of the training dataset S .
 - In general, the more *complex* $\mathcal{H}$, the larger the error.
 - The training dataset error can be reduced by increasing its size. If dataset is not large enough we get a wide gap ϵ between EmpLoss and TrueLoss.

$$|EmpLoss(S, h^*) - TrueLoss(h^*)| = \epsilon$$

- **Main Takeaway:** it is *generally* not safe to rely on $EmpLoss(S, h^*)$ value to determine if model is good.
- What is alternative? And how can we detect overfitting?
 - We searched through $\mathcal{H}$ to find the best h^* for T , so performance on T *tends* to be good.
 - We can use another dataset Z that was not used for training. This is typically called the **validation set** or **test set**.
 - Compute $EmpLoss(Z, h^*)$. This is a better estimate of $TrueLoss(h^*)$, and its only source of error is the size of Z .
 - You can detect overfitting if $EmpLoss(T, h^*)$ is low, but $EmpLoss(Z, h^*)$ is high.

Training and Test sets

- Suppose we have gathered a labelled dataset S of size m from a real-world process.
- Split the dataset S into two sets: Training set T and test set Z .
- Note: other names for test set are validation set and held-out set. Some ML references denote the validation set as being different from test set (so you have three sets: training, validation, and test).
- Suppose we use an ML algorithm that considers a hypothesis space with 100 functions $\mathcal{H} = \{h_1, h_2, \dots, h_{100}\}$
- Train the algorithm to select from $\mathcal{H}$ the loss minimizer h^* on the training set T :

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(T, h)$$

Note use of
training set

- Note that since the training set T was used to select the hypothesis from $\mathcal{H}$. The empirical loss of h^* on T is biased to be low. This loss is called the **training loss**.
- So, we need another labelled dataset that has not been used in the training. We can use the test set Z . This is the **test loss**, and it is an *unbiased* (good) estimator of the true loss.

$$EmpLoss(Z, h^*) \approx TrueLoss(h^*)$$

Supervised Learning Summary

- A supervised learning problem with input x and output y is modelled as a probability model where the sampling follows some order like $x \sim D$, then $y|x \sim Q$; this is $p(x, y) = p(x)p(y|x)$. Other formulations are $p(x, y) = p(y)p(x|y)$ or directly modelling the joint $p(x, y)$
- We do not know underlying distributions. But have access to *sampled* dataset $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$
- Most ML algorithms will make assumptions on the relationship between x and y . These assumptions give rise to a hypothesis space $\mathcal{H}$. Some hypothesis spaces are more complex than others (able to learn complex decision boundaries/surfaces).
- When training, the ML algorithm will minimize some loss function over the training set and find $h^* \in \mathcal{H}$. This is called empirical loss minimization (ERM).
- The training loss is not a good estimate of the true loss. It has two sources of error: the hypothesis space and training set size.
- Unless you have a sufficiently large training set that covers the underlying data distribution, overfitting can occur if we rely on the training loss.
- So, it is recommended, that you use another dataset's loss (not the training set). We call this test set.

An ML Algorithm with a hypothesis set $\mathcal{H}$ is said to **overfit** the training data T , if the training loss minimizer $h^* \in \mathcal{H}$ has a training loss that is far from the true loss of the underlying data distribution. That is, if h^* overfits, then:

$$|EmpLoss(T, h^*) - TrueLoss(h^*)| \geq \epsilon$$

This says that the difference between the training loss and the true loss is too high for us to accept. The acceptance threshold is set via the value ϵ . The larger the training set, the less this is likely to happen. The more complex the hypothesis set is, the more likely this will happen because we can find a complex enough hypothesis to fit the training data exactly.

- Remember we cannot detect this because we don't know the true underlying distribution, that is we cannot compute $TrueLoss(h^*)$.
- So, we use another, unbiased dataset not used in training, like the test set Z to get a better estimate of the true loss of h^* . We use the test set loss to see if it is acceptable for our learning task.

Supervised Learning Summary – Overfitting and Underfitting in Practice

- A hypothesis h is said to **overfit** a training set, if the training set loss is low ($EmpLoss(T, h)$ is low) and the true loss is high ($TrueLoss(h)$ is high).
 - A practical way to detect this is if the training set loss is low and the test set loss is high.
- A hypothesis h is said to **underfit** a training set, if the training set loss is high ($EmpLoss(T, h)$ is high) and the true loss is high ($TrueLoss(h)$ is high).
 - A practical way to detect this is if the training set loss is high and the test set loss is high.
- The notions of *high* and *low* is set by you depending on your problem. You can formalize it by specifying an acceptance tolerance/threshold value ϵ .

Practical Consideration – Splitting, training, and evaluating

Splitting the dataset Train-Test set

- Suppose your labelled dataset is S .
- You want to split S into training set T and test set Z .
- A large training set gives more examples to learn from.
- A large test set gives better estimate of true loss/performance.
- Rule of thumb in practice: split S such that train is 80% and test is 20%. E.g. for 100 examples, train is 80 examples and test is 20 examples.
- How should we split the data?
 - Regression task: split should be **uniformly random**.
 - Classification task: The split should **stratified random**.

Splitting the dataset Train-Test: Regression Task

Labelled dataset

$$\mathcal{S} = \{ (x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_{100}, y_{100}) \}$$

80-20 split
uniformly
random

How to split uniformly at random?

In python:

`np.random.permute(100)`

Training dataset randomly selects 80 instances

$$\mathcal{T} = \{ (x_{95}, y_{95}), (x_{12}, y_{12}), (x_{45}, y_{45}), \dots, (x_{71}, y_{72}) \}$$

Test dataset randomly selects 20 instances

$$\mathcal{Z} = \{ (x_{81}, y_{81}), (x_{65}, y_{65}), \dots, (x_{32}, y_{32}) \}$$

- Note that these two sets are disjoint. In other words, no instances from $\mathcal{Z}$ are in $\mathcal{T}$, and vice-versa.

Splitting the dataset Train-Test: Classification Task

Labelled dataset with 30 **+** labels and 70 **-** labels.

$$\mathcal{S} = \{ (x_1, +), (x_2, -), (x_3, -), \dots, (x_{100}, +) \}$$

80-20 split
stratified
random

Split randomly, but try to preserve the same percentage of **+** and **-**. Why?

How to do this in python?

Training dataset randomly selects 80 instances

Ensure there are 24 **+** labels and 56 **-** labels

$$\mathcal{T} = \{ (x_{95}, -), (x_{12}, +), (x_{45}, +), \dots, (x_{71}, -) \}$$

Test dataset randomly selects 20 instances

Ensure 6 **+** labels and 14 **-** labels

$$\mathcal{Z} = \{ (x_{81}, +), (x_{65}, +), \dots, (x_{32}, -) \}$$

- Note that these two sets are disjoint. In other words, no instances from $\mathcal{Z}$ are in $\mathcal{T}$, and vice-versa.

Splitting, training, and evaluating using [scikit-learn](#)

- Suppose your labelled dataset is S consists of 1000, and you want to split into 80% training and 20% testing.
- See [link](#)

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import accuracy_score
5
6 # stratify split the dataset X, y into 80-20 train-test
7 X_train, X_test, y_train, y_test = train_test_split(X, y,
8                                                     test_size=0.2,
9                                                     stratify=y,
10                                                     random_state=42)
11
12 # train a linear regression model
13 model = LinearRegression()
14 model.fit(X_train, y_train)
15
16 # evaluate accuracy on test set
17 y_pred_on_test = model.predict(X_test)
18 test_acc = accuracy_score(y_true=y_test, y_pred=y_pred_on_test)
19 print('Test accuracy: {}'.format(test_acc))
```

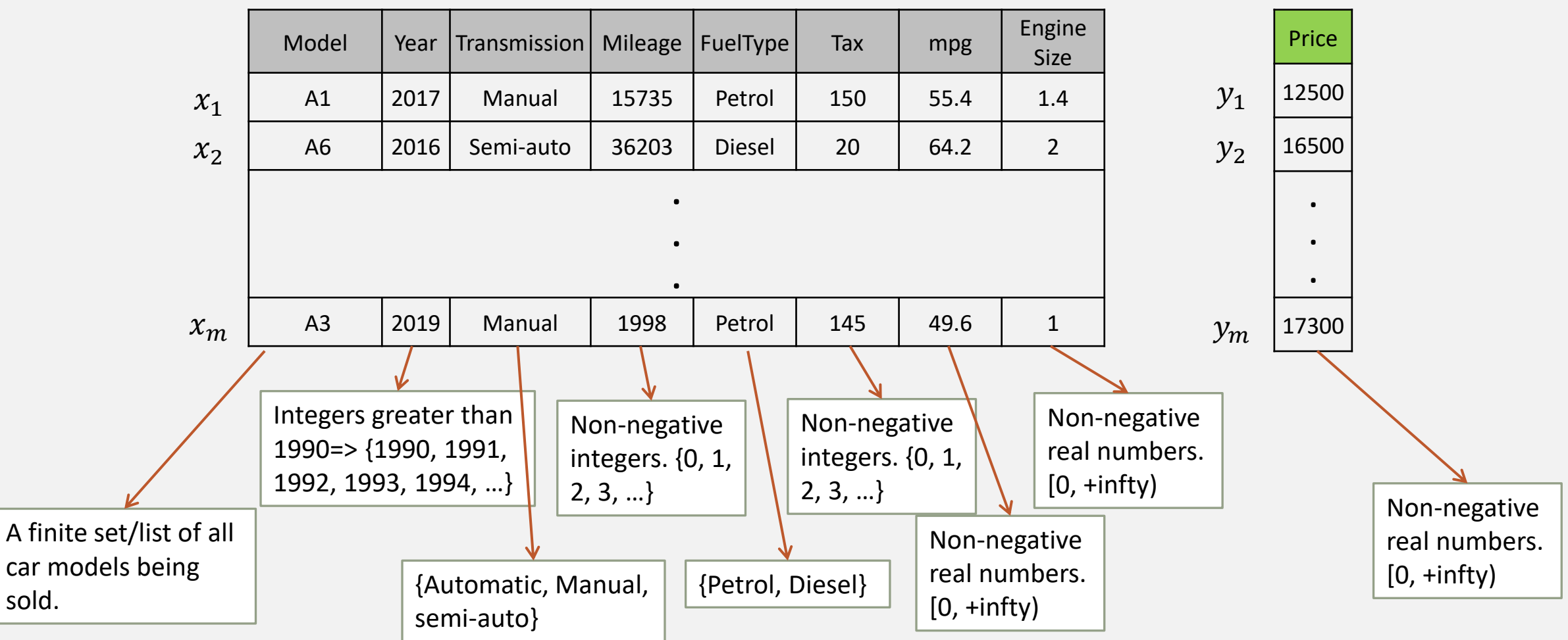
Practical Consideration – Data Preprocessing

Dataset Feature Pre-processing [1]

- Before training an ML algorithm, the labelled dataset is **pre-processed** and prepared for input based on the requirements of the algorithm.
- Some algorithms require that all columns are numbers. So how do we change string columns?
- If the string column only has a finite set of possible values, then you can map each string value to a unique number.

Dataset Feature Pre-processing [2]

Suppose this is $m=1000$ labelled examples. You can see it has $n=8$ feature dimensions. Can you define suitable domains for the feature columns and a range for the output?



Dataset Feature Pre-processing [3]

	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	A1	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	A6	2016	Semi-auto	36203	Diesel	20	64.2	2	y_2	16500
				.						.
				.						.
				.						.
x_m	A3	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

The Model column takes on values from a finite set/list of all car models being sold. We can map each unique model to unique integer:
BMW → 1, Accord → 2, A1 → 3, A2 → 4, A3 → 5, A4 → 6, A5 → 7, A6 → 8
etc.

Dataset Feature Pre-processing [4]

	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	3	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	8	2016	Semi-auto	36203	Diesel	20	64.2	2	y_2	16500
			.	.	.					.
			.	.	.					.
			.	.	.					.
x_m	5	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

- The Transmission column takes on 3 values {Automatic, Manual, Semi-auto} which we can map to 0, 1, 2 (or any three numbers).
- However, mapping them to numbers implies there is an ordering between Automatic, Manual, and Semi-auto. Even if there is an ordering like in {small, medium, large} how far is the ordering distance?
- A better mapping would be one-hot encoding; each value is mapped to a unique bit-vector.
- Automatic $\rightarrow$ [1, 0, 0], Manual $\rightarrow$ [0, 1, 0], Semi-auto $\rightarrow$ [0, 0, 1] there is no ordering, but rather an on-off bit to indicate the transmission type.

Dataset Feature Pre-processing [5]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size	
x_1	3	2017	0	1	0	15735	Petrol	150	55.4	1.4	y_1
x_2	8	2016	0	0	1	36203	Diesel	20	64.2	2	y_2
					.						.
					.						.
					.						.
x_m	5	2019	1	0	0	1998	Petrol	145	49.6	1	y_m

Price

12500

16500

.

.

.

17300



The FuelType column takes on 2 values {Petrol, Diesel} which we can map to 0 and 1 which is similar to one-hot encoding [1, 0] and [0,1]. Since both mappings are on-off bits. However, in practice, 1-bit vs on-hot-encoding of a binary variable can result in different models for some algorithms.

Dataset Feature Pre-processing [6]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size	
x_1	3	2017	0	1	0	15735	0	150	55.4	1.4	y_1
x_2	8	2016	0	0	1	36203	1	20	64.2	2	y_2
	• • •										
x_m	5	2019	1	0	0	1998	0	145	49.6	1	y_m

Price
12500
16500
•
•
•
17300

Now all inputs (features) and outputs are numbers.

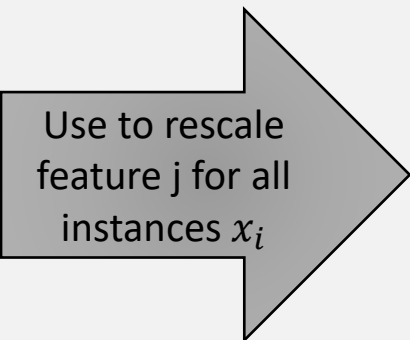
Note pre-processing is an involved topic with many techniques, we might not spend a lot of time discussing it. You can go from here to learn more on your own.

Dataset Feature Pre-processing [6]: Feature rescaling/normalization

- Some algorithms *benefit* from having the **continuous** input features on the same range of values. We will see this in k -nearest neighbors, linear regression, logistic regression, neural networks.
- Import notes:
 - Any rescaling technique you use should be based on training set T only.
 - You use the training set to derive the *parameters* of the rescaling technique.
 - Those *same* parameters are then used for rescaling training, validation, test, and new instances.
- **Min-max Scaler (Normalization)**: rescales the feature j to be in a range $[a, b]$ (typically $[0,1]$).

$$\min_j := \arg \min_{x_i \in T} x_{ij}$$

$$\max_j := \arg \max_{x_i \in T} x_{ij}$$



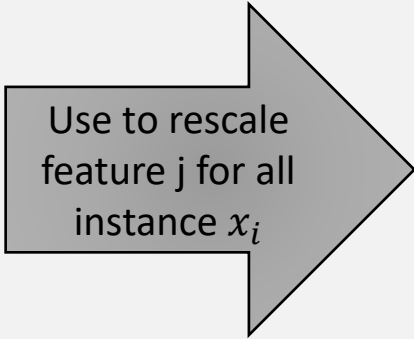
Use to rescale
feature j for all
instances x_i

$$x_{ij} := \frac{x_{ij} - \min_j}{\max_j - \min_j} (b - a) + a$$

Note min
and max of
training set.

Dataset Feature Pre-processing [7]: Feature rescaling/normalization

- **Standard Scaler**: rescales the feature j to be centered around 0 and with 1 standard deviation (0 mean and 1 standard deviation).

$$\hat{\mu}_j := \frac{1}{|T|} \sum_{x_i \in T} x_{ij}$$
$$\hat{\sigma}_j := \sqrt{\frac{1}{|T| - 1} \sum_{x_i \in T} (x_{ij} - \hat{\mu}_j)^2}$$


Use to rescale feature j for all instance x_i

$$x_{ij} := \frac{x_{ij} - \hat{\mu}_j}{\hat{\sigma}_j}$$

Note mean and deviation compute from training set

- Primary reason we rescale numeric features is to put them on relatively the same scale so that they can be compared. Example, we have feature 1 with range $[-1000, 1000]$ and feature 2 with range $[0.05, 0.8]$.
- The output label in regression tasks can also be rescaled for easier training of some algorithms. More on this in later lectures.

Dataset Feature Pre-processing [8]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	3	2017	0	1	0	15735	0	150	55.4	1.4	y_1	12500
x_2	8	2016	0	0	1	36203	1	20	64.2	2	y_2	16500
					.							.
					.							.
					.							.
x_m	5	2019	1	0	0	1998	0	145	49.6	1	y_m	17300

- We commonly apply feature scaling for continuous features. So, for this dataset, we would apply feature scaling to: Year, Mileage, Tax, mpg, and Enginesize.
- The feature Model can also be rescaled, but it *may* be better to use one-hot encoding to remove ordering.
- Finally, Price can also be rescaled if the values have a massive range that would result in issues when training neural networks [advanced topic].

Min-Max and Standard Scaler Example

Feature
24.32
18.44
-58.51
-33.04
16.96
88.01
-3.3
65.99
6.82
76.64

	Column stats
mean	20.23
stdev	46.75
min	-58.51
max	88.01

Min-Max Scaled Feature
0.57
0.53
0.00
0.17
0.52
1.00
0.38
0.85
0.45
0.92

	Column stats
mean	0.54
stdev	0.32
min	0
max	1

Standard Scaled Feature
0.09
-0.04
-1.68
-1.14
-0.07
1.45
-0.50
0.98
-0.29
1.21

	Column stats
mean	0
stdev	1
min	-1.68
max	1.45

Dataset Feature Pre-processing in python

- Scikit-learn contains great support for preprocessing, evaluation metrics, and general ML utility functions.
- **Note:** fit the scaler on the training data to derive parameters.
- Test set is transformed by the scaler (no fitting).
- See more info for preprocessing in scikit-learn [here](#).

```
import pandas as pd
import numpy as np
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import accuracy_score
```

```
# read data from csv
data_df = pd.read_csv('mydata.csv')
X_data = data_df[['feat1', 'feat2', 'feat3', 'feat4', 'feat5']]
y_data = data_df['label']
```

```
# split data into train and test using scikit-learn
# stratify arg used when stratify sampling
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data,
                                                    test_size=0.25, stratify=y_data,
                                                    random_state=42)
```

```
# fit scaler on training data
scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# fit LR model
lr_model = LogisticRegression()
lr_model.fit(X_train_scaled, y_train)
```

```
# evaluate on test data
y_pred_test = lr_model.predict(X_test_scaled)
test_acc = accuracy_score(y_test, y_pred_test)
print(f'Test accuracy is {test_acc}')
```

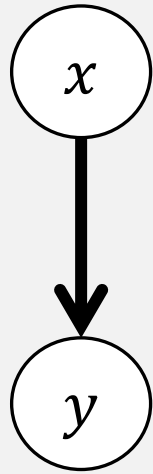
Generative vs Discriminative

ML Algorithms We'll Cover

- What we'll (try) to cover:
 - Naïve Bayes (used for classification)
 - Linear regression (used for regression)
 - Logistic regression (used for classification)
 - Neural networks (used for classification and regression)
 - K-nearest neighbors (used for classification and regression)
 - Decision trees (used mainly for classification, but can be adapted to regression)
 - Ensemble methods (used for classification and regression)

Generative vs Discriminative

Probabilistic dependency graph
(or Bayesian Network)



First, x_i is sampled from some vector distribution $x \sim D$.

Then, y_i given x_i is sampled from some conditional distribution $y|x \sim Q$

This is done for each $i = 1, 2, \dots, m$

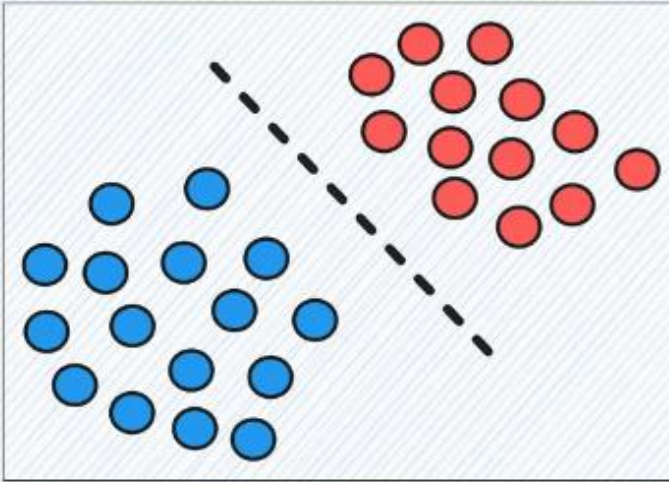
This says that $p(x, y) = p(x)p(y|x)$ where $x \sim D$ and $y|x \sim Q$

- Recall the supervised learning probability setup.
- ML algorithms can be categorized as **generative** or **discriminative** based on their modelling of $p(x, y)$

Generative vs Discriminative

- ML algorithms can be categorized as **generative** or **discriminative** based on their modelling of $p(x, y)$
- A generative algorithm is concerned with both input x and output y , and attempts to approximate $p(x, y)$
 - By modelling $p(x, y)$, you can generate samples (x, y) using the learned model.
 - This is why it's called generative, as you can use it to *generate* complete samples (x, y)
 - Note that these models can also be used for predicting y given x .
- A discriminative algorithm is concerned with just predicting the label y *given* input x ; it attempts to approximate $p(y | x)$
 - By modelling $p(y | x)$, you can generate (predict) y **given** x , but you cannot generate samples for x .
 - In essence, the sample x has already been generated, your model handles predicting $y | x$
 - This is why it's called discriminative, because you use model to *discriminate* the labelling given input.

Discriminative Models



Discriminative models don't possess generative properties

Logistic Regression

Random Forests

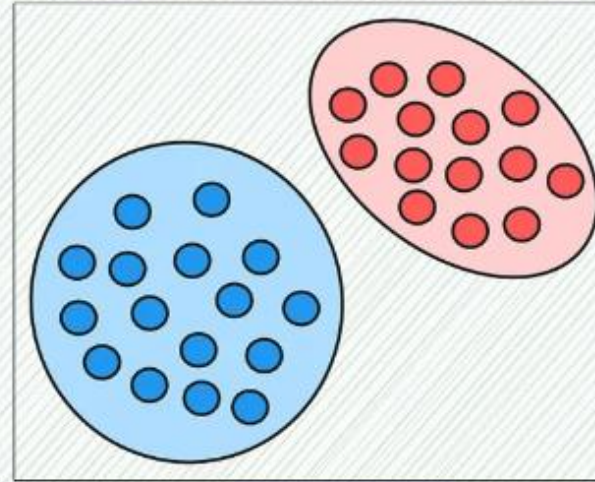
SVMs

Neural Networks

Decision Tree

kNN

Generative Models



Generative models possess discriminative properties

Hidden Markov Models

Naive Bayes

Gaussian Mixture Models

Gaussian Discriminant Analysis

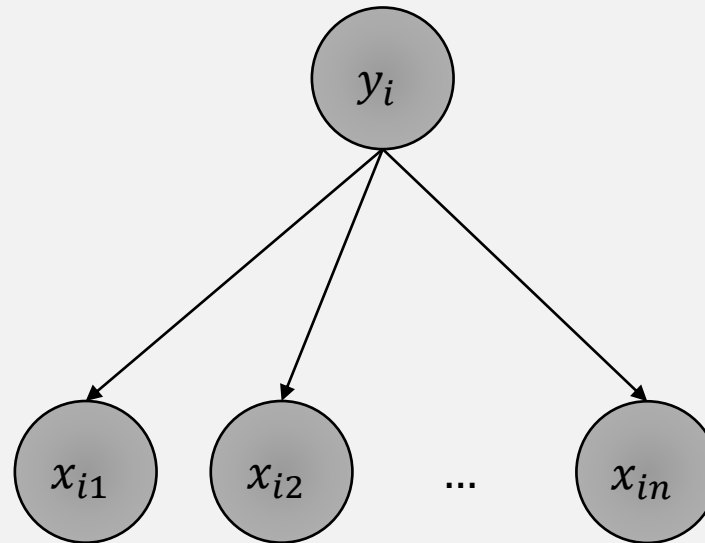
LDA

Bayesian Networks

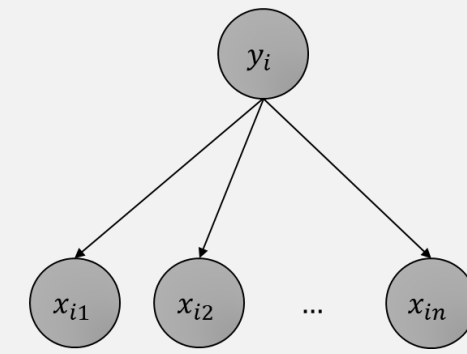
Naïve Bayes for Supervised Learning

Naïve Bayes for Supervised Learning – Overview

- Naïve bayes is primarily used for classification tasks.
- It is a generative model. It models $p(x, y) = p(y)p(x | y)$; so y is sampled, then $x | y$ is sampled.
- We *make the assumption* that the input features are conditionally independent of each other given the label.
- Given a sample x_i with n features, we get $p(x_i, y_i) = p(y_i)p(x_{i1} | y_i)p(x_{i2} | y_i) \dots p(x_{in} | y_i)$
- So, the sampling order is y , then each feature is sampled given y .
- See BN illustration below.



Naïve Bayes for Supervised Learning – Overview



- Given a sample x_i with n features, the probability that the label y_i belongs to class c is:

$$P(y_i = c \mid x_i) = \frac{P(x_i, y_i = c)}{P(x_i)} = \frac{P(x_i \mid y_i = c)P(y_i = c)}{P(x_i)}$$

$$= \frac{1}{P(x_i)} P(x_{i1}, x_{i2}, \dots, x_{in} \mid y_i = c) P(y_i = c)$$

Apply assumption: the features are **conditionally independent** of each other given the label.

$$= \frac{1}{P(x_i)} P(x_{i1} \mid y_i = c) P(x_{i2} \mid y_i = c) \dots P(x_{in} \mid y_i = c) P(y_i = c)$$

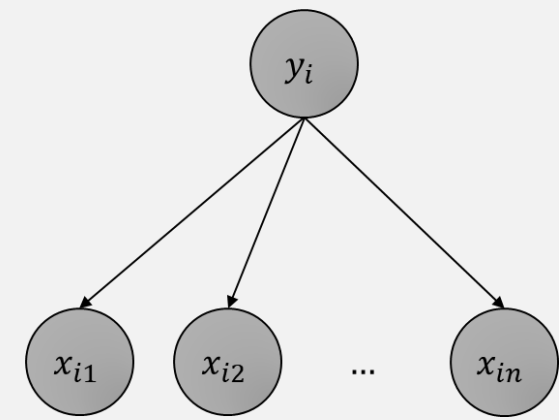
$$= \frac{1}{P(x_i)} P(y_i = c) \prod_{q=1}^n P(x_{iq} \mid y_i = c)$$

↑

This is called the “**prior**”. Typically, if we have K classes, the prior of each is set to $1/K$ (uniform/equal probability of each class). However, you can also set it using domain knowledge or estimate it from data.

Naïve Bayes for Supervised Learning – Training

- Consider a K -class problem where input x has n features and output $y \in \{1, 2, \dots, K\}$
- Suppose you have a training set $T = \{(x_1, y_1), \dots, (x_m, y_m)\}$
- How to train a naïve bayes?
- From the sampling structure, we are modelling $p(y_i) \prod_j p(x_{ij} \mid y_i)$
 - We can model each variable using a common distribution.
 - Model $y_i \sim \text{Cat}(p_1, p_2, \dots, p_K)$ estimates the parameters from the training set.
 - Model each feature $x_{ij} \mid y_i$ as coming from a distribution that is appropriate for the feature type.
 - For discrete features we can use Bernoulli or Categorical
 - For continuous features we can use Normal
 - Estimate parameters of the distribution using the training set.
- So, training a NB is estimating the parameters of the assumed distributions using the training set.
- We can use the trained NB to generate samples (generative model), and predict labels.



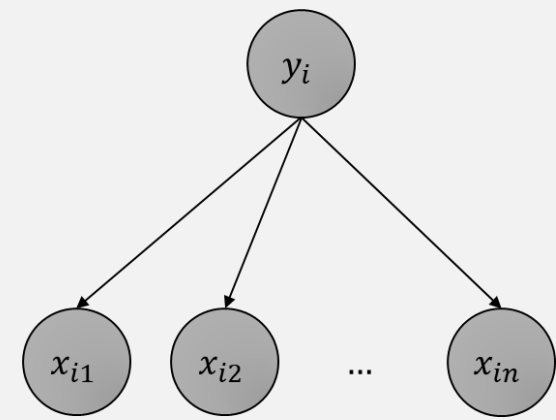
Naïve Bayes for Supervised Learning – Prediction

- After training, we can compute $p(y_{new} = c \mid x_{new})$ using the estimated parameters.
- To estimate the class label $\hat{y}_{new}$ given x_{new} , we select the class that gives the *highest* probability.

$$\hat{y}_{new} = \operatorname{argmax}_{c \in \{1, \dots, K\}} p(y = c \mid x_{new})$$

$$= \operatorname{argmax}_{c \in \{1, \dots, K\}} \frac{p(y = c) \prod_{j=1}^n p(x_{new,j} \mid y = c)}{p(x_{new})}$$

$$= \operatorname{argmax}_{c \in \{1, \dots, K\}} p(y = c) \prod_{j=1}^n p(x_{new,j} \mid y = c)$$



We can remove $1/P(x_{new})$ since it is a constant with respect to the argmax parameter.

Naïve bayes with discrete features example [1]

$$\hat{y}_{new} = \operatorname{argmax}_{c \in \{1, \dots, K\}} p(y = c) \prod_{j=1}^n p(x_{new,j} \mid y = c)$$

- Consider the following training set.
- We can model as follows:
 - $play \sim \text{Bernolli}(p_{yes})$
 - $outlook \mid play \sim \text{Cat}(p_{sunny}, p_{overcast}, p_{rain})$
 - $temp \mid play \sim \text{Cat}(p_{hot}, p_{mild}, p_{cool})$
 - $humidity \mid play \sim \text{Cat}(p_{normal}, p_{high})$
 - $wind \mid play \sim \text{Cat}(p_{weak}, p_{strong})$

Training set

outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Naïve bayes with discrete features example [2]

- Use the training set to estimate the parameters.

$$P(\text{Play}=\text{Yes}) = 9/14 \quad P(\text{Play}=\text{No}) = 5/14$$

$P(\text{Outlook} \mid \text{Play})$

Outlook	Play=Yes	Play=No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rain	3/9	2/5

$P(\text{Temperature} \mid \text{Play})$

Temperature	Play=Yes	Play=No
Hot	2/9	2/5
Mild	4/9	2/5
Cool	3/9	1/5

$P(\text{Humidity} \mid \text{Play})$

Humidity	Play=Yes	Play=No
High	3/9	4/5
Normal	6/9	1/5

$P(\text{Wind} \mid \text{Play})$

Wind	Play=Yes	Play=No
Strong	3/9	3/5
Weak	6/9	2/5

$$\hat{y}_{new} = \underset{c \in \{1, \dots, K\}}{\operatorname{argmax}} \quad p(y = c) \prod_{j=1}^n p(x_{new,j} \mid y = c)$$

Training set

outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Naïve bayes with discrete features example [3]

- Use Naïve Bayes to predict the label for

$x_{new} = [Sunny, Cool, High, Strong]$

- Using the estimated parameters we can now predict for each class.

$$P(Yes | x_{new}) \propto P(Sunny | Yes)P(Cool | Yes)P(High | Yes)P(Strong | Yes)P(Play=Yes) = 0.0053$$

$$P(No | x_{new}) \propto P(Sunny | No)P(Cool | No)P(High | No)P(Strong | No)P(Play=No) = 0.0206$$

- Thus, $y_{new} = No$ since its probability is largest.

$$\hat{y}_{new} = \operatorname{argmax}_{c \in \{1, \dots, K\}} p(y = c) \prod_{j=1}^n p(x_{new,j} | y = c)$$

$$P(\text{Play}=Yes) = 9/14 \quad P(\text{Play}=No) = 5/14$$

$P(\text{Outlook} | \text{Play})$

Outlook	Play=Yes	Play=No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rain	3/9	2/5

$P(\text{Temperature} | \text{Play})$

Temperature	Play=Yes	Play=No
Hot	2/9	2/5
Mild	4/9	2/5
Cool	3/9	1/5

$P(\text{Humidity} | \text{Play})$

Humidity	Play=Yes	Play=No
High	3/9	4/5
Normal	6/9	1/5

$P(\text{Wind} | \text{Play})$

Wind	Play=Yes	Play=No
Strong	3/9	3/5
Weak	6/9	2/5

Slide adapted from [link](#)

Additive Smoothing

$$p(y = c) \prod_{j=1}^n p(x_{new,j} | y = c)$$

Training set

Shape	Color	Y
Square	Red	No
Square	Green	No
Triangle	Red	Yes
Square	Green	Yes
Square	Red	Yes
Square	Green	No
Triangle	Red	Yes

- Consider the given training set. Suppose that:
 - $Shape \in \{Square, Triangle\}$
 - $Color \in \{Red, Green, Blue\}$ notice that there is no blue occurrence in training set.
 - $y \in \{No, Yes, Maybe\}$ notice that there is no maybe occurrence in training set.
- What we have used so far in this course to estimate discrete parameters for Bernoulli and Categorical is frequency-based. $\hat{p}(e) = \frac{\# \text{ samples with } (e)}{\# \text{ total samples}}$
- However, if a feature value never occurs in the training data, then the frequency-based probability estimate will be zero. Two things to consider:
 - It could be that this *specific* training set was small (or unlucky) that it did not gather a value for that feature. So, the true probability should not be zero.
 - More problematic with naïve bayes, is that it will zero-out out other probabilities when they are multiplied. In this example, $\hat{p}(Color = Blue | y = No) = 0$
- Therefore, it is often desirable to incorporate a small-sample correction, so that probability estimates are not exactly zero. This is known as pseudocounts, additive-smoothing, or Lidstone smoothing.

Additive Smoothing

- Consider discrete variable with K outcomes. If the frequency of outcome i is n_i out of m samples, then additive-smoothed probability of outcome i is:

$$\hat{p} = \frac{n_i + \alpha}{m + \alpha K}$$

- α is called the pseudocount. When $\alpha = 0$, no correction/smoothing. When $\alpha = 1$, it is called Laplace smoothing. For other values, it is generally called additive smoothing or Lidstone smoothing.
- Here is the estimated parameters for the given training set using Laplace smoothing ($\alpha = 1$).

Training set

Shape	Color	Y
Square	Red	No
Square	Green	No
Triangle	Red	Yes
Square	Green	Yes
Square	Red	Yes
Square	Green	No
Triangle	Red	Yes

$Shape \in \{Square, Triangle\}$
 $Color \in \{Red, Green, Blue\}$
 $y \in \{No, Yes, Maybe\}$

$P(y = Yes)$	$P(y = No)$	$P(y = Maybe)$
$\frac{4 + 1}{7 + 3}$	$\frac{3 + 1}{7 + 3}$	$\frac{0 + 1}{7 + 3}$

$P(Color | y)$

Color	Y=Yes	Y=No
Red	$\frac{3 + 1}{4 + 3}$	$\frac{1 + 1}{3 + 3}$
Green	$\frac{1 + 1}{4 + 3}$	$\frac{2 + 1}{3 + 3}$
Blue	$\frac{0 + 1}{4 + 3}$	$\frac{0 + 1}{3 + 3}$

$P(Shape | y)$

Shape	Y=Yes	Y=No
Square	$\frac{2 + 1}{4 + 2}$	$\frac{3 + 1}{3 + 2}$
Triangle	$\frac{2 + 1}{4 + 2}$	$\frac{0 + 1}{3 + 2}$

Naïve bayes with continuous features example [1]

- Consider the 4-class label dataset.
- Notice we have two continuous features, so we consider continuous distributions.
- We'll assume that $P(x_{ij} \mid y = c)$ is a normal distribution $Normal(\mu, \sigma^2)$.
- We need to estimate μ and σ^2 for all 4 classes.
- Recall the normal density function
$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}}$$
- The sample mean formula $\hat{\mu} = \frac{1}{n} \sum_{x_i} x_i$
- The sample variance formula $\widehat{\sigma^2} = \frac{1}{n-1} \sum_{x_i} (x_i - \hat{\mu})^2$
- $P(y = 0) = 0.25, P(y = 1) = 0.25, P(y = 2) = 0.25, P(y = 3) = 0.25$

class	feature 1 $\hat{\mu}$	feature 1 $\widehat{\sigma^2}$	feature 2 $\hat{\mu}$	feature 2 $\widehat{\sigma^2}$
0	10.07	3.34	7.88	3.76
1	21.14	5.06	19.43	5.77
2	9.36	4.74	18.80	5.93
3	21.45	1.66	9.85	3.33

Training set

Feature 1	Feature 2	output
22.85	19.44	1
18.60	21.17	1
23.40	16.62	1
10.41	5.98	0
9.36	6.49	0
20.94	9.98	3
21.96	17.57	1
13.10	7.26	0
7.37	19.24	2
12.53	20.24	2
8.59	9.02	0
22.96	8.08	3
8.89	10.67	0
9.29	20.02	2
22.66	8.03	3
10.30	14.49	2
7.34	20.00	2
18.88	22.37	1
19.98	10.93	3
20.73	12.24	3

Naïve bayes with continuous features example [2]

class	feature 1 mean	feature 1 variance	feature 2 mean	feature 2 variance
0	10.07	3.34	7.88	3.76
1	21.14	5.06	19.43	5.77
2	9.36	4.74	18.80	5.93
3	21.45	1.66	9.85	3.33

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2} \frac{(x-\mu)^2}{\sigma^2}}$$

- Predict for the following: $x_{new} = [9.5, 18]$

$$P(y = 0 \mid x_{new}) \propto P(y = 0)P(feat1 = 9.5 \mid y = 0)P(feat2 = 18 \mid y = 0)$$

Note: recall that for continuous variables, $P(A=value) = 0$; that is, we need an interval. However, since we are concerned with max-probability, we can assume we are using a small *fixed* interval centered around our point. So, this allows us to treat this interval as constant and drop it from the calculation all together (since we just want max-probability).

$$P(feat1 = 9.5 \mid y = 0) \propto \frac{1}{\sqrt{2\pi 3.34}} e^{-\frac{1}{2} \frac{(9.5-10.07)^2}{3.34}} = 0.208 \qquad P(feat2 = 18 \mid y = 0) \propto \frac{1}{\sqrt{2\pi 3.76}} e^{-\frac{1}{2} \frac{(18-7.88)^2}{3.76}} = 2.504 \times 10^{-7}$$

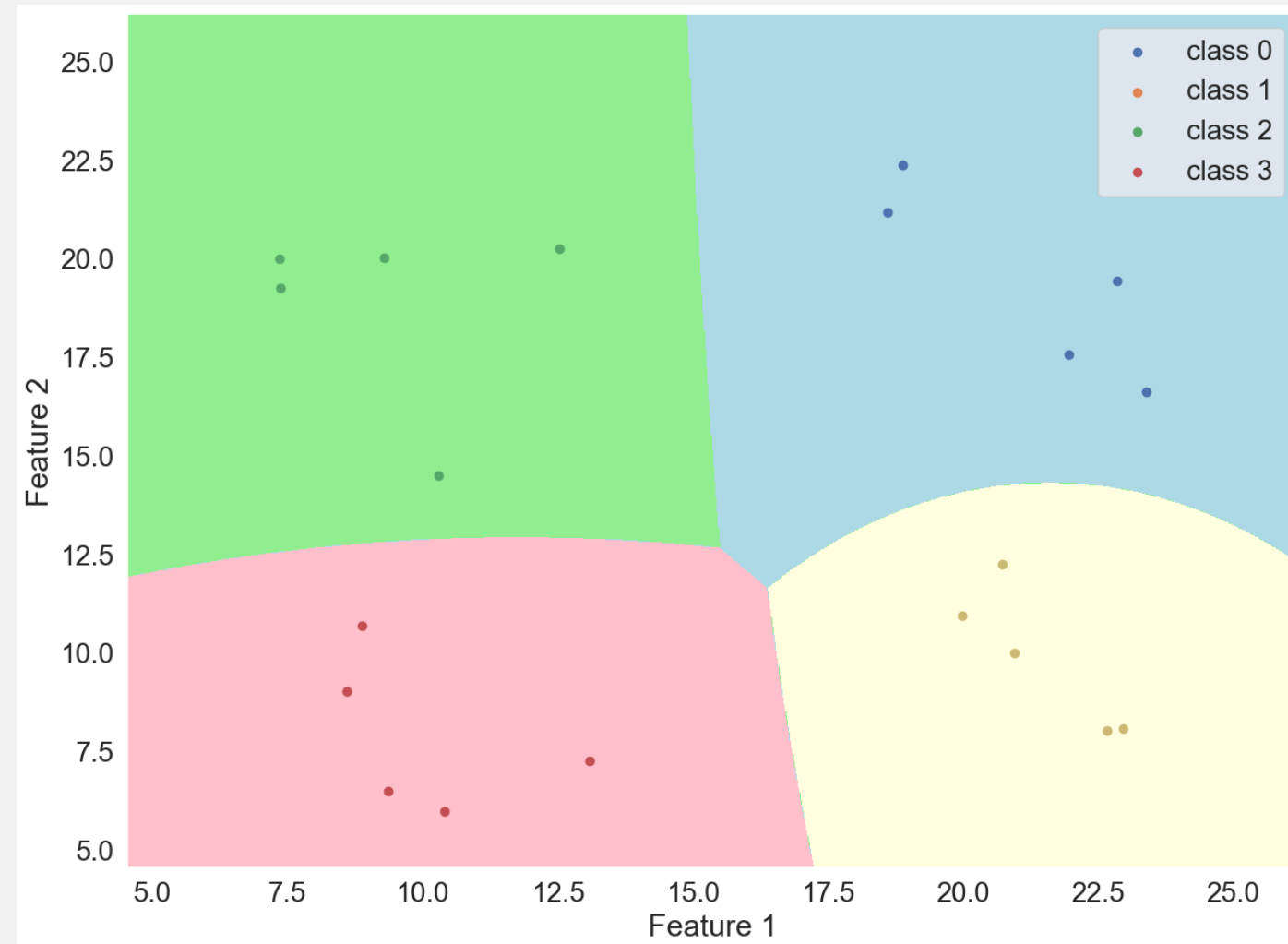
$$P(y = 0 \mid x_{new}) \propto 0.25 \times 0.208 \times (2.504 \times 10^{-7}) = 1.3 \times 10^{-8}$$

$$P(y = 1 \mid x_{new}) \propto 9.45 \times 10^{-9} \qquad P(y = 2 \mid x_{new}) \propto 0.0071 \qquad P(y = 3 \mid x_{new}) \propto 1.65 \times 10^{-25}$$

Class 2 has highest probability, so we predict 2.

Naïve bayes with continuous features example [3]

- If we plot the decision boundary for this classifier, we get the following.
- Naïve bayes is not necessarily a linear classifier. The shape depends on the distributions used and the spread of the data.
- As can be seen from the plot, the point $x_{new} = [9.5, 18]$ is within class 2 subspace.
- With naïve bayes we can have a mix of discrete and continuous features. Just assume a suitable distribution for each feature and estimate parameters using the data.
- What about if our label is continuous?



Naïve Bayes scikit-learn

```
class sklearn.naive_bayes.CategoricalNB(*, alpha=1.0, force_alpha=True,  
fit_prior=True, class_prior=None, min_categories=None) \[source\]
```

```
class sklearn.naive_bayes.GaussianNB(*, priors=None,  
var_smoothing=1e-09) \[source\]
```

- Scikit-learn Naïve Bayes for classification problems:
 - [BernoulliNB](#) assumes all your features are binary.
 - [CategoricalNB](#) assumes all your features are categorical.
 - [GaussianNB](#) assumes all your features are continuous features and models them as normal distribution.
 - See others here [link](#).
- What to do if you have mix of discrete and continuous? You can train two algorithms GaussianNB and CategoricalNB. Predict probabilities using the two models. Multiply the two probabilities to get a final probability.

Naïve Bayes Discussion

- Naïve bayes easy to implement, fast computationally, scales well with number of features and dataset size.
- Historically it was used with great effect for text classification problems (e.g., spam detection, sentiment analysis).
- Can handle missing data.
- Generative model. Can be used to generate samples.
- Disadvantages:
 - In practice, independence assumption does not hold, which can reduce model performance.
 - Probability Calibration: The predicted probabilities provided by Naive Bayes can be poorly calibrated, which means they might not always reflect the true likelihood of class membership. This is why it is used for predicting labels, not predicting probabilities.
 - Limited to Classification: Naive Bayes is inherently a classification algorithm and does not handle regression tasks directly.